

The Hitchhiker's Compiler: un compilador de HULK en Rust con genericidad, interfaces y recolección de basura

Kevin Alejandro Torres Perera, Lianny de la Caridad Revé Valdivieso, and
Jocdan Lismar López Mantecón

Grupos 311 y 312, Ciencias de la Computación,
Facultad de Matemática y Computación,
Universidad de La Habana

<https://github.com/MatCom-Developing-Team-10/The-Hitchhikers-Compiler>



Resumen Este trabajo presenta *The Hitchhiker's Compiler*, un compilador completo para el lenguaje HULK (*Havana University Language for Kompilers*) implementado en Rust como proyecto integrador de las asignaturas Compilación y Lenguajes de Programación. El compilador cubre el pipeline clásico: análisis léxico, sintáctico LALR(1), análisis semántico con verificación de tipos, descenso a una representación intermedia y ejecución sobre una máquina virtual de pila, organizado como un *workspace* de siete *crates* con dependencias unidireccionales que materializan los principios SOLID y DRY. El análisis semántico se estructura en una secuencia de pasadas —cuatro fundamentales más inferencia de tipos y verificación de interfaces— para satisfacer la visibilidad global de declaraciones que exige HULK, acumula diagnósticos en lugar de abortar y emplea un patrón *poison* (`Type::Error`) para suprimir cascadas de errores. Sobre esta base, el equipo diseñó, implementó y validó tres extensiones al lenguaje, fundamentadas en una comparación con lenguajes de programación conocidos: (i) **genericidad** con parámetros de tipo [T] resueltos por *type erasure* e invarianza, como en Java y Scala; (ii) **interfaces** con cláusulas `implements/extends` y conformidad tanto nominal como estructural, combinando los modelos de Java y Go; y (iii) un **recolector de basura** *mark-and-sweep* con *heap* indexado por *handles* que reclama ciclos. Para cada extensión se dan reglas de tipado formales, su realización en el código y una comparación cualitativa con lenguajes de referencia.

Keywords: compiladores, HULK, Rust, sistemas de tipos, polimorfismo paramétrico, interfaces, recolección de basura, máquina virtual

1. Introducción

La construcción de un compilador es, posiblemente, el ejercicio integrador por excelencia de las Ciencias de la Computación: reúne teoría de lenguajes formales, estructuras de datos, sistemas de tipos, diseño de software y arquitectura de máquinas en un único artefacto verificable. Un *compilador* es un programa que traduce texto escrito en un lenguaje fuente a un lenguaje objeto, preservando su significado [2]. Clásicamente, la traducción se descompone en un *frontend* —análisis léxico, sintáctico y semántico, que comprende el programa y rechaza los incorrectos— y un *backend* —síntesis, que genera código o lo ejecuta— unidos por una o varias representaciones intermedias.

Este informe documenta *The Hitchhiker's Compiler*, un compilador del lenguaje HULK desarrollado en Rust. El proyecto persigue un doble objetivo académico. Desde la asignatura **Compilación**, demuestra el dominio del pipeline completo, desde el flujo de caracteres hasta la ejecución, con un reporte de errores posicionado y la conformidad con el contrato de evaluación automática de la cátedra (*matcom/compilers*). Desde la asignatura **Lenguajes de Programación**, justifica y materializa una extensión original del lenguaje, cuyas decisiones de diseño se sustentan en la comparación científica con lenguajes históricos estudiados en el curso.

1.1. El lenguaje HULK

HULK (*Havana University Language for Kompilers*) es un lenguaje didáctico, fuertemente tipado, orientado a objetos con herencia simple, y de naturaleza *expresional*: prácticamente toda construcción —incluidos `if` y bloques— produce un valor [11]. Un programa HULK es una secuencia de declaraciones de tipos y funciones seguida de una expresión principal. El lenguaje ofrece literales numéricos, booleanos y de cadena; operadores aritméticos, booleanos con cortocircuito, de comparación y de concatenación (`@`, `@@`); enlaces locales `let ... in`; condicionales `if/elif/else`; ciclos `while` y `for`; funciones de primer orden y recursión; declaración de tipos con constructores parametrizados, atributos, métodos con despacho virtual, herencia (`inherits`) y despacho al padre (`base`); y los operadores de tipo `is` y `as`.

1.2. Objetivo y alcance

Los objetivos son: (a) construir un compilador correcto y comprobable para HULK que cubra las especificaciones exigidas, y (b) diseñar extensiones del lenguaje con reglas de tipado precisas, implementarlas sobre la arquitectura existente y compararlas con las decisiones de lenguajes conocidos. Las tres extensiones son:

1. **Genericidad** (polimorfismo paramétrico): parámetros de tipo con sintaxis `[T]` resueltos por *type erasure*, con conformidad invariante. Se contrastan con el *erasure* de Java y la monomorfización de Rust.

2. **Interfaces:** contratos de métodos con herencia múltiple entre interfaces (*extends*) y conformidad que admite tanto la vía nominal (*implements*) como la estructural (*protocol-style*). Se sitúan entre el modelo nominal de Java y el estructural de Go.
3. **Recolector de basura** *mark-and-sweep* con *heap* indexado por *handles*, capaz de reclamar ciclos, en la tradición del *tracing* iniciada por McCarthy en Lisp.

El resto del documento se organiza como sigue. La Sección 2 desarrolla el marco teórico que sustenta el proyecto. La Sección 3 describe la arquitectura y la implementación de cada fase del compilador. La Sección 4 —núcleo del aporte original— define, analiza y demuestra las tres extensiones. La Sección 5 resume la estrategia de pruebas y calidad, y la Sección 7 ofrece conclusiones, comparativas, críticas y trabajo futuro.

2. Marco teórico

2.1. Anatomía de un compilador

Siguiendo a Aho et al. [2], la traducción se modela como una secuencia de fases que transforman progresivamente la representación del programa. El *análisis léxico* agrupa los caracteres en *tokens*; el *análisis sintáctico* reconoce la estructura gramatical y construye un árbol de sintaxis abstracta (AST); el *análisis semántico* verifica reglas que la gramática no captura —sobre todo el sistema de tipos y la resolución de nombres—; y la *síntesis* genera código intermedio y final. La Figura 3 muestra la instancia concreta de este esquema en nuestro proyecto.

2.2. Análisis léxico y autómatas finitos

El analizador léxico se especifica con expresiones regulares, cuyo reconocimiento equivale a un autómata finito determinista (AFD). Generadores modernos como *logos* [25] traducen anotaciones declarativas sobre un tipo enumerado a un AFD eficiente en tiempo de compilación, evitando el costo de construir el autómata en tiempo de ejecución. La salida es un flujo de *lexemas* clasificados, cada uno acompañado de su posición en la fuente, base indispensable para los diagnósticos.

2.3. Análisis sintáctico LR(1) y LALR(1)

La sintaxis de los lenguajes de programación se describe con gramáticas libres de contexto. Entre los métodos de análisis ascendente, las gramáticas LR(1) [18] reconocen una clase amplia de lenguajes deterministas con un símbolo de anticipación. Las gramáticas **LALR(1)** [9] fusionan estados LR(1) con el mismo núcleo, reduciendo drásticamente el tamaño de la tabla de análisis a cambio de una pérdida mínima de poder expresivo; este es el equilibrio que adoptan generadores como *yacc*, *bison* y, en el ecosistema Rust, *LALRpop* [20]. La *precedencia* y la *asociatividad* de los operadores se resuelven estratificando la gramática en niveles, técnica que evita ambigüedades sin recurrir a directivas externas.

```

Algoritmo 1: LCA( $A, B$ )
anc  $\leftarrow \{A\} \cup \{\text{ancestros de } A \text{ hasta } \text{Object}\}$ 
para cada  $C$  en la cadena  $B, \text{padre}(B), \dots, \text{Object}$ :
    si  $C \in \text{anc}$  entonces devolver  $C$ 
devolver  $\text{Object}$       (siempre alcanzable: es el supremo)

```

Figura 1. Cálculo del ancestro común más bajo (*join* en el retículo de herencia).

2.4. El árbol de sintaxis abstracta

El AST es la columna vertebral del compilador: una representación en memoria, independiente de la sintaxis concreta, que las fases posteriores recorren y anotan. Un buen diseño de AST equilibra *fidelidad* (preservar la información necesaria, incluidos los *spans* de origen para diagnosticar) y *economía* (no arrastrar detalles redundantes). En lenguajes algebraicos como Rust, el AST se modela idiomáticamente con tipos suma (`enum`), lo que hace del recorrido un *pattern matching* exhaustivo y verificado por el compilador.

2.5. Sistemas de tipos, conformidad y subtipado

Un *sistema de tipos* es un método sintáctico tratable para demostrar la ausencia de ciertos comportamientos erróneos clasificando las frases del lenguaje según los valores que computan [23]. En un lenguaje orientado a objetos con herencia, la relación central es la de *subtipado* o *conformidad*: $S \leq T$ significa que un valor de tipo S puede usarse en todo contexto que espere un T (principio de sustitución de Liskov [19]). HULK organiza sus tipos en una jerarquía de herencia simple con un tipo tope `Object`; la conformidad se calcula recorriendo la cadena de ancestros. Cuando una expresión compuesta como `if` tiene ramas de tipos distintos, su tipo es el *ancestro común más bajo* (LCA) de las ramas, análogo a la noción de *join* en un retículo de tipos.

Formalmente, la relación de conformidad $\leq$ que implementa nuestro verificador (función `conforms`) se rige por las reglas de la Figura 2. La regla (TOP) hace de `Object` el supremo; (REFL) y (TRANS-INH) dan reflexividad y el ascenso por la cadena de herencia; (POISON) convierte a `Type::Error` en comodín bidireccional para suprimir cascadas; y (VEC-ITER) expresa que un vector conforma al iterable tipado covariantemente en el elemento. Las reglas de genericidad e interfaces (INV y IFACE) se desarrollan en la Sección 4.

El cálculo del LCA de dos tipos A y B se reduce a recolectar la cadena de ancestros de A (incluido él mismo) hasta `Object` y devolver el primer ancestro de B que pertenezca a ese conjunto; como la herencia es simple y acíclica, el procedimiento siempre termina y, en el peor caso, en `Object` (Algoritmo 1). Esta operación es la que fija el tipo de un `if` con ramas heterogéneas y el tipo de elemento de un literal de vector.

(TOP) $T \leq \mathbf{Object}$	(REFL) $T \leq T$	(POISON) $\mathbf{Error} \leq T \wedge T \leq \mathbf{Error}$
(TRANS-INH) $\frac{U \text{ inherits } V \quad V \leq T}{U \leq T}$	(VEC-ITER) $\frac{S \leq T}{S[] \leq T*}$	

Figura 2. Reglas de conformidad $\leq$ del verificador de tipos (núcleo A.8.4). Las reglas para genericidad (invarianza) e interfaces (nominal/estructural) aparecen en la Sección 4.

2.6. Polimorfismo

Cardelli y Wegner [7] distinguen el polimorfismo *universal* —paramétrico (genericidad) y de inclusión (subtipado)— del *ad-hoc* —sobrecarga y coerción—. El **polimorfismo paramétrico** permite escribir código uniforme sobre una familia de tipos mediante variables de tipo. Existen dos estrategias de realización:

- **Borrado de tipos** (*type erasure*): los parámetros de tipo existen solo en compilación y se eliminan antes de ejecutar. Es la vía de Java [6] y mantiene una única representación en tiempo de ejecución, a costa de perder información de tipo en *runtime*.
- **Monomorfización**: el compilador genera una copia especializada del código para cada instanciación concreta. Es la vía de C++ y Rust [17]: más rápida y con información de tipo completa, pero a costa de mayor tamaño de código.

2.7. Subtipado nominal frente a estructural

Un sistema **nominal** establece la relación de subtipo solo cuando se declara explícitamente (`implements`, `extends`): el nombre manda. Un sistema **estructural** la deduce de la forma: un tipo conforma a una interfaz si posee los miembros requeridos, haya o no declaración. Java, Kotlin y Swift son nominales; Go y TypeScript son estructurales. El primero evita acoplamientos accidentales y produce mensajes más claros; el segundo favorece el desacoplamiento y la composición. Como se verá, nuestra extensión de interfaces ofrece *ambas* disciplinas de forma controlada.

2.8. Representación intermedia y máquinas de pila

Entre el frontend y la ejecución, los compiladores introducen una representación intermedia (IR) que desacopla el lenguaje fuente de la máquina objetivo [3]. Una *máquina de pila* es un modelo de ejecución en el que los operandos se empujan y extraen de una pila; su IR es un *bytecode* compacto y fácil de generar a partir de un AST mediante un recorrido post-orden, razón por la cual la adoptan intérpretes como los de Python, la JVM o Lua [22]. Es el modelo que elegimos para el *backend*.

2.9. Gestión automática de memoria

La *recolección de basura* (GC) automatiza la liberación de memoria inalcanzable [14]. Existen dos grandes familias:

Conteo de referencias. Cada objeto mantiene un contador de referencias entrantes; al llegar a cero se libera. Es incremental y simple, pero **no reclama ciclos** por sí solo, lo que obliga a añadir un detector de ciclos como el de Bacon y Rajan [4] (la estrategia de CPython).

Rastreo (*tracing*). Se parte de un conjunto de *raíces* (pila, variables globales) y se visitan transitivamente los objetos alcanzables; lo no alcanzado es basura. El algoritmo **mark-and-sweep**, introducido por McCarthy para Lisp [21,8], recorre y marca lo vivo (*mark*) y luego barre lo muerto (*sweep*). Reclama ciclos de forma natural, a costa de pausas *stop-the-world*.

Refinamientos posteriores —copia semiespacial, recolección generacional [15] y colectores concurrentes como los de Go y la JVM— pueden mejorar la latencia y el rendimiento. Lenguajes como Rust prescinden de GC mediante un sistema de *ownership* verificado estáticamente [17]; HULK, al permitir mutación compartida a través de objetos, sí requiere gestión automática.

3. Arquitectura y desarrollo

3.1. Organización del *workspace*

El compilador es un *workspace* de Cargo con siete *crates*, conectados por el flujo de datos `lexer -> ast <- parser -> semantic -> ir -> vm` y orquestados por el binario `hulkc` (Figura 3). La dirección de las dependencias es estrictamente unidireccional: ninguna fase posterior puede contaminar a una anterior, lo que encarna el principio de inversión de dependencias y permite razonar sobre cada fase de forma aislada.

Cuadro 1. Responsabilidad de cada *crate* del *workspace*.

Crate	Responsabilidad
<code>hulk-ast</code>	Nodos del AST, <code>Span</code> , operadores y declaraciones (contrato compartido).
<code>hulk-lexer</code>	Tokenización con <i>logos</i> ; adapta el flujo al protocolo de LALRpop.
<code>hulk-parser</code>	Gramática LALRpop $\rightarrow$ <code>Program</code> (AST).
<code>hulk-semantic</code>	Análisis semántico: nombres, <i>scopes</i> , herencia, tipos.
<code>hulk-ir</code>	Descenso del AST a <i>bytecode</i> de máquina de pila.
<code>hulk-vm</code>	Intérprete de <i>bytecode</i> con GC <i>mark-and-sweep</i> .
<code>hulkc</code>	<i>Driver</i> de línea de comandos; orquesta el pipeline.

El *crate* `hulk-ast` ocupa una posición central: tanto el parser que lo produce como el analizador semántico y el generador de IR que lo consumen dependen

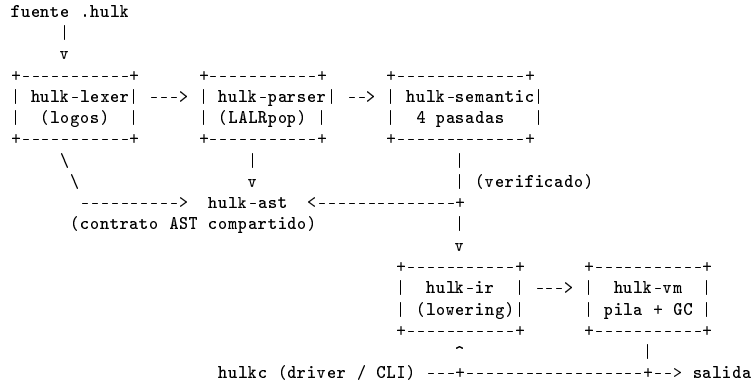


Figura 3. Pipeline de *The Hitchhiker's Compiler*. El AST (`hulk-ast`) es el contrato compartido entre el frontend y las fases posteriores; las dependencias fluyen en una única dirección.

de él. Mantener el AST como única fuente de verdad evita la duplicación de definiciones y materializa el principio DRY: un nodo nuevo se define una sola vez y todas las fases lo ven de inmediato.

3.2. Análisis léxico (`hulk-lexer`)

El lexer se construye sobre *logos*, que genera un AFD eficiente a partir de anotaciones declarativas sobre un `enum Token`. Reconoce medio centenar de entradas léxicas (cerca de cincuenta reglas `##[token]/##[regex]`), incluidos los patrones de descarte de espacios y comentarios: palabras clave (`let`, `in`, `if`, `elif`, `else`, `while`, `for`, `function`, `type`, `inherits`, `new`, `is`, `as`, `true`, `false`, y las de las extensiones `interface`, `implements`, `extends`), operadores —incluidos los de concatenación `@` y `@@` y el de potencia—, delimitadores (entre ellos `[` y `]` añadidos para genericidad y vectores), literales e identificadores. Una decisión relevante es tratar `self` y `base` como identificadores ordinarios, dejando su semántica especial a fases posteriores. El lexer descarta espacios y comentarios de línea (`//`); no soporta comentarios de bloque. Expone un `Lexer<input>` que adapta el iterador de logos al protocolo *triple* (`inicio`, `Token`, `fin`) que LALRpop exige, y representa los errores con `LexError`, que guarda los *offsets* de byte del fragmento errado.

3.3. Análisis sintáctico (`hulk-parser`)

El parser usa LALRpop. La gramática completa vive en `grammar.lalrpop` y se traduce a Rust mediante un *build script*. La API pública es una única función `parse(source) -> Result<Program, ParseError>` que invoca el lexer internamente, de modo que el consumidor solo interactúa con texto y un AST. La

gramática define **doce niveles de precedencia**, desde la asignación destructiva `:=` en el nivel más bajo hasta el acceso a miembros y las llamadas en el más alto; el operador de potencia `^` es asociativo por la derecha y los unarios tienen mayor precedencia que la multiplicación pero menor que la potencia, conforme a la especificación. Dos decisiones merecen mención. El `let` con múltiples enlaces se *desazucara* a `Let` unarios anidados, eliminando toda lógica de enlace paralelo del verificador. Y el `if` lleva siempre una rama `else` obligatoria: como en HULK el `if` es una expresión que debe producir un valor en toda rama, una rama faltante haría su tipo indefinible. Cada nodo carga un `Span` (par de *offsets* lo/hi) capturado con los marcadores `@L/@R` de LALRpop. El `let` múltiple se desazucara en el parser: `let a = 1, b = a + 1 in c` se traduce a `let a = 1 in (let b = a + 1 in c)`, una cadena de `let` unarios cuyo alcance anidado reproduce la semántica secuencial de A.4.1 (un enlace ve a los anteriores) y simplifica el verificador.

3.4. El AST (`hulk-ast`)

El AST define un `Program` raíz que agrupa declaraciones de interfaces, tipos y funciones, más una expresión principal. Las expresiones se modelan con un `enum ExprKind` de más de veinte variantes (literales, operaciones binarias y unarias, variables, bloques, `let`, `if/elif/else`, `while`, `for`, llamadas a funciones y métodos, acceso y asignación de atributos, `new`, `is/as`, y los vectores `Vector`, `VectorComp` e `Index`). Cada `Expr` combina su `ExprKind` con un `Span`; los *offsets* de byte se convierten a línea/columna solo en el punto de reporte, evitando arrastrar información redundante por todo el árbol.

3.5. Análisis semántico (`hulk-semantic`)

El análisis semántico es el corazón del compilador y se organiza como una **secuencia de ocho pasadas** sobre el AST (Tabla 2), articulada en torno a cuatro pasadas *fundamentales* (`collect`, `sign`, `check_overrides`, `check_bodies`) a las que se intercalan tres pasadas de **inferencia de tipos** (A.9) y una de verificación de **interfaces** (extensión). El diseño multi-pasada responde a un requisito de HULK: todas las funciones y tipos son visibles con independencia del orden de declaración, por lo que es imposible verificar cuerpos en una sola pasada.

Las tres pasadas de inferencia —`infer_ctor_params`, `infer_params` e `infer_returns`, ejecutadas tras `sign`— recuperan anotaciones omitidas (A.9), una capacidad que excede el mínimo A.1–A.8. Conviene ser preciso sobre su alcance, porque *no* se trata de inferencia al estilo Hindley–Milner: no hay variables de tipo ni unificación [23]. El mecanismo es **inferencia local dirigida por el uso**, en la línea de la *local type inference* de Pierce y Turner [24], con dos formas:

- **Parámetros sin anotar** (`infer_ctor_params`, `infer_params`): cada parámetro recibe una *restricción* a partir de sus usos en el cuerpo —por ejemplo, al pasarse como argumento donde se espera un tipo conocido, o al invocar

sobre él un método que pertenece a un único tipo—. Las restricciones se combinan por su ancestro común (LCA); si un parámetro no queda restringido, su tipo es `Object`.

- **Tipos de retorno** (`infer_returns`): un **punto fijo acotado** sobre el grafo de dependencias entre firmas. En cada iteración se re-verifica el cuerpo de cada función/método sin retorno anotado (con los errores suprimidos durante la inferencia) y se actualiza su tipo de retorno; el proceso se repite mientras alguna firma cambie, con un tope de $(\#inferibles + 1)$ iteraciones.

El bucle **siempre termina** por el tope de iteraciones. Cuando el grafo de dependencias es **acíclico**, cada iteración resuelve al menos un nivel, de modo que se alcanza el tipo local de cada firma. La iteración es necesaria justamente porque el retorno de una función puede depender del de otra aún sin resolver; una sola pasada no basta. Para **recursión mutua** entre funciones sin anotar (p. ej. f llama a g y g a f) no garantizamos converger al tipo ideal dentro del tope —al no haber unificación, el resultado puede quedar en un tipo conservador o en `Object`—; esos casos quedan fuera de lo que exige A.9 y se resuelven anotando explícitamente el retorno.

Cuadro 2. Las ocho pasadas del análisis semántico tal como las orquesta `analyze()`: cuatro fundamentales, tres de inferencia y una de interfaces (extensión).

# Pasada	Propósito
1 <code>collect</code>	Registrar nombres de tipos; rechazar duplicados, herencia de <i>builtins</i> y ciclos.
2 <code>sign</code>	Llenar firmas de métodos y funciones; resolver tipos de parámetros anotados.
3 <code>infer_ctor_params</code> (A.9.3)	Inferir tipos de parámetros de constructor sin anotar.
4 <code>infer_params</code>	(A.9) Inferir tipos de parámetros de función/método sin anotar.
5 <code>infer_returns</code>	Inferir tipos de retorno sin anotar (por punto fijo).
6 <code>check_interfaces</code>	(extensión) Verificar que cada <code>implements</code> cubre los métodos requeridos.
7 <code>check_overrides</code>	Verificar que los <i>overrides</i> mantienen firma idéntica al padre.
8 <code>check_bodies</code>	Recorrer expresiones; inferir/verificar tipos; acumular errores.

Una decisión central es el **manejo de errores por acumulación**: las pasadas nunca hacen *panic*; recogen todos los diagnósticos en un `Vec<SemError>` y los retornan juntos, lo que permite reportar múltiples problemas en una sola ejecución. Cada variante de `SemError` (más de quince: variable indefinida, tipo no coincidente, aridad incorrecta, nombre reservado, firma de *override* inconsistente, `NotAnInterface`, `NotIndexable`, etc.) carga un `Span` accesible por `span()`.

Listing 1.1. Algunas variantes de `SemError`; cada una porta su `Span` para diagnosticar con precisión.

```
#[derive(Debug, Error)]
pub enum SemError {
    #[error("undefined variable '{name}'")]
    UndefinedVariable { name: String, span: Span },
    #[error("type mismatch: expected '{expected}', found '{found}'")]
    Mismatch { expected: String, found: String, span: Span },
    #[error("wrong number of arguments: expected {expected}, found {found}")]
    Arity { expected: usize, found: usize, span: Span },
    #[error("method '{name}' overrides parent with a different signature")]
    OverrideSignatureMismatch { name: String, span: Span },
    // ... (cyclic inheritance, duplicate type, no such method, etc.)
}
```

Para evitar **cascadas de errores**, el verificador usa el patrón *poison* `Type::Error`. Cuando una subexpresión falla, recibe el tipo `Type::Error`, que la relación de conformidad trata como comodín: conforma con cualquier tipo y cualquier tipo conforma con él. Así, un único error profundo no genera docenas de errores derivados. El `enum Type` (Listado 1.2) modela los tipos primitivos, `Object`, los tipos de usuario, y —por las extensiones— `Generic`, `Param`, `Iterable` y `Vector`.

Listing 1.2. El `enum Type` con las variantes de las extensiones.

```
pub enum Type {
    Number, String, Boolean, Object,
    User(String), // tipo de usuario no generico
    Generic(String, Vec<Type>), // List[Number], Map[String, Point]
    Param(String), // parametro de tipo 'T' sin resolver
    Iterable(Box<Type>), // T* (A.11.2)
    Vector(Box<Type>), // T[] (A.12)
    Error, // poison: suprime cascadas
}
```

3.6. Representación intermedia y máquina virtual

Tras la verificación, `lower_program` desciende el AST a un *bytecode* para una máquina de pila. El *crate hulk-vm* es un intérprete que ejecuta las instrucciones sobre una pila de operandos y soporta el lenguaje completo: aritmética (con potencia y módulo), booleanos con cortocircuito, comparaciones, concatenación, control de flujo (`if/elif/else`, `while`, `for` sobre el protocolo iterable `next()/current()`), recursión con detección de desbordamiento de pila, orientación a objetos (herencia, constructores parametrizados, despacho virtual, `self`, `base`), y funciones *builtin* (`sqrt`, `sin`, `cos`, `exp`, `log`, `rand`, `print`, `range`). Para soportar recursión profunda, el programa se ejecuta en un hilo dedicado con una pila de gran tamaño.

Cuadro 3. Conjunto de instrucciones del IR (`enum Instr`), agrupado por función.

Grupo	Instrucciones
Constantes	<code>PushNum</code> , <code>PushBool</code> , <code>PushStr</code> , <code>PushNil</code>
Pila	<code>Pop</code> , <code>Dup</code>
Aritmética	<code>Add</code> , <code>Sub</code> , <code>Mul</code> , <code>Div</code> , <code>Pow</code> , <code>Mod</code> , <code>Neg</code>
Lógica/comp.	<code>And</code> , <code>Or</code> , <code>Not</code> , <code>Eq</code> , <code>Ne</code> , <code>Lt</code> , <code>Le</code> , <code>Gt</code> , <code>Ge</code>
Cadenas	<code>Concat</code> , <code>ConcatWs</code>
Variables	<code>LoadVar</code> , <code>StoreVar</code> , <code>BeginScope</code> , <code>BindVar</code> , <code>EndScope</code>
Control	<code>Label</code> , <code>Jump</code> , <code>JumpIfFalse</code> , <code>Call</code> , <code>Ret</code>
Builtins	<code>Print</code> , <code>Sqrt</code> , <code>Sin</code> , <code>Cos</code> , <code>Exp</code> , <code>Log</code> , <code>Rand</code>
Objetos	<code>NewObject</code> , <code>GetField</code> , <code>SetField</code> , <code>CallMethod</code> , <code>CallBase</code>
Tipos	<code>IsType</code> , <code>AsType</code>
Vectores	<code>MakeVector</code> , <code>Index</code> , <code>VecPush</code>

El conjunto de instrucciones (`enum Instr`) es deliberadamente pequeño y ortogonal (Tabla 3). Las constantes y las operaciones aritméticas, booleanas y de comparación trabajan exclusivamente sobre la pila; el control de flujo se expresa con etiquetas simbólicas (`Label`) y saltos (`Jump`/`JumpIfFalse`); los *scopes* léxicos se delimitan con `BeginScope`/`EndScope` y los enlaces con `BindVar`; y la orientación a objetos se reduce a `NewObject`, `GetField`/`SetField`, `CallMethod` (despacho dinámico por el tipo concreto) y `CallBase` (despacho estático al método del padre). Esta uniformidad hace que el descenso sea un recorrido post-orden directo del AST.

Costo del despacho Es justo precisar el costo de este modelo, que no está optimizado. El despacho dinámico (`CallMethod`) resuelve el método en cada llamada: a partir del `type_name` del objeto receptor, `resolve_method` recorre la cadena de herencia consultando un `HashMap` de métodos por tipo hasta hallar la primera definición. No hay *vtables* ni *inline caching* [2]: el costo por llamada es $O(d)$ búsquedas en tabla, con d la profundidad de herencia (no una búsqueda lineal sobre el *heap*). Igualmente, los objetos y vectores guardan sus campos en un `HashMap<String, Value>`, cómodo de implementar pero lejos del *layout* contiguo que usaría un compilador serio. Estas son decisiones conscientes a favor de la claridad sobre el rendimiento.

A modo de ilustración, la expresión principal del Listado 1.4 —`print((new Dog("Toby")).speak())`— desciende al *bytecode* del Listado 1.3. Nótese cómo `CallMethod` difiere la resolución al *runtime* (el objeto en la cima de la pila determina la implementación de `speak`), mientras que dentro de `Dog::speak` la llamada `base()` se compila como un `CallBase` que nombra estáticamente al tipo padre.

Listing 1.3. Descenso a *bytecode* de la expresión principal del Listado 1.4.

```

NewObject "Dog"          ; aloca Dog en el heap -> ObjectId en la pila
PushStr   "Toby"         ; argumento del constructor
CallMethod "<ctor>", 1    ; inicializa los campos
CallMethod "speak", 0    ; despacho dinamico por el type_name del objeto
Print                                           ; emite el String resultante

```

3.7. El *driver* y el contrato de evaluación (hulkc)

El binario `hulkc` orquesta el pipeline e implementa el contrato de `matcom/compilers/interface.md`. La regla `make build` compila en modo *release* y copia el artefacto a `./hulk`. La invocación `./hulk archivo.hulk` valida las tres fases y, en caso de éxito, genera un ejecutable `./output`, `./output` se materializa como un *script* autocontenido que embebe la fuente validada y re-invoca el binario `hulk` en un modo interno `exec`; esta decisión reutiliza íntegramente el *backend* y satisface el contrato sin la complejidad de un generador a C o LLVM. Los errores se imprimen a `stderr` con el formato exacto `(line,col) TYPE: message`, con `TYPE ∈ {LEXICAL, SYNTACTIC, SEMANTIC}`, y el código de salida (1, 2 o 3) refleja el error más fundamental encontrado.

3.8. Un ejemplo de extremo a extremo

Para ilustrar cómo cooperan las fases, considere el programa del Listado 1.4, que combina herencia, despacho virtual y `base`.

Listing 1.4. Programa HULK que ejercita herencia y despacho al padre.

```

type Animal(name: String) {
  name: String = name;
  speak(): String => self.name @ " hace un ruido";
}
type Dog(name: String) inherits Animal(name) {
  speak(): String => base() @ " ... guau!";
}
print((new Dog("Toby")).speak()); // Toby hace un ruido ... guau!

```

El flujo es el siguiente. (1) El **lexer** produce los tokens `type`, `Animal`, `(`, `...`, clasificando `inherits` y `base` y anotando cada uno con su *span*. (2) El **parser** construye el `Program` con dos `TypeDecl` y la expresión principal; la concatenación `@` se anida según su nivel de precedencia y `base()` se representa como una llamada especial al método homónimo del padre. (3) El **análisis semántico** registra `Animal` y `Dog` en `collect`, llena sus firmas en `sign`, comprueba en `check_overrides` que `Dog::speak` tiene la misma firma `() : String` que `Animal::speak`, y en `check_bodies` verifica que `self.name` es `String`, que `base()` resuelve a `Animal::speak` y que el argumento de `print` conforma a lo esperado. (4) El **descenso a IR** genera *bytecode* de pila: instrucciones para

instanciar `Dog`, invocar `speak` con despacho dinámico y resolver `base` estáticamente al método del padre. (5) La **VM** ejecuta el *bytecode*: aloca el objeto `Dog` en el *heap* (devolviendo un `ObjectId`), despacha `speak` por el `type_name` concreto, ejecuta la concatenación y emite la salida `Toby hace un ruido ... guau!`. Si el programa tuviera un error —por ejemplo, si `Dog::speak` declarara un retorno `Number`— la fase 3 lo reportaría como (6,5) **SEMANTIC: method speak overrides parent with a different signature** y el proceso terminaría con código 3, sin alcanzar el *backend*.

4. Extensiones al lenguaje

Esta sección define, analiza y demuestra las tres extensiones. Todas se implementaron como *commits* independientes y *backwards-compatible*: todo programa HULK estándar sigue funcionando sin cambios. La Tabla 4 sintetiza las decisiones de diseño y sus referentes históricos.

Cuadro 4. Resumen de las extensiones y sus referentes en lenguajes históricos.

Extensión	Estrategia	Referente
Genericidad	<i>type erasure</i> , invarianza	Java, Scala (<i>vs</i> Rust: monomorfización)
Interfaces	nominal + estructural	Java (nominal) + Go (estructural)
GC	<i>mark-and-sweep</i> , <i>handles</i>	Lisp, Lua (<i>vs</i> Python: <i>refcount</i>)

4.1. Genericidad: polimorfismo paramétrico por borrado de tipos

Definición y sintaxis. Un parámetro de tipo se declara entre corchetes tras el nombre del tipo o función y se usa como cualquier otro tipo en parámetros, atributos y retornos. La instanciación aporta los argumentos de tipo concretos:

Listing 1.5. Tipos y funciones genericidad en HULK.

```
type Box[T](item: T) {
  item: T = item;
  get(): T => self.item;
  set(v: T): T => self.item := v;
}
type Pair[A, B](a: A, b: B) { a: A = a; b: B = b; }
function id[T](x: T): T => x;

let n: Number = (new Box[Number](42)).get() in print(n); // 42
```

Reglas semánticas. El sistema introduce dos variantes de tipo: `Type::Generic(name, args)` para una instancia como `List[Number]`, y `Type::Param(name)` para un parámetro `T` dentro del cuerpo de una declaración genérica. Las reglas son:

- **Invarianza:** `List[Animal]` *no* conforma a `List[Object]`. Se exige igualdad exacta de argumentos. Esta elección evita las sorpresas de varianza en un lenguaje didáctico y coincide con el comportamiento de Java para genericidad crudos.
- **Borrado en runtime:** `new List[Number]()` se ejecuta como `new List()`; la VM no ve los argumentos de tipo.
- **Chequeo de aridad:** un número incorrecto de argumentos de tipo produce un error `Arity` en compilación.
- **Sustitución:** en `(new Box[Number](42)).get()`, el retorno `T` se sustituye por `Number` mediante `substitute`.
- **Type::Param conforma solo a sí mismo y a Object.**

Implementación. El lexer aporta `[/]`; el AST añade `TypeRef::Generic(String, Vec<TypeRef>)` y un campo `generic_params` en `TypeDecl` y `FunctionDecl`; el parser introduce las reglas `GenericParams/GenericArgs`; y la semántica añade `resolve_type_ref_in_scope` y `substitute`. El IR y la VM *no* cambian estructuralmente, justamente porque el borrado de tipos hace que la representación en ejecución sea independiente de los argumentos genericidad.

Sustitución en acción. La regla de sustitución es la que da utilidad práctica a los genericidad. Al verificar `(new Box[Number](42)).get()`, el analizador instancia el cuerpo de `Box[T]` con el mapa $\{T \mapsto \text{Number}\}$: el atributo `item: T` pasa a `Number`, el constructor exige un argumento `Number` (y 42 conforma), y el retorno `get(): T` se reescribe a `get(): Number`. Así, la expresión completa recibe tipo `Number` y el enlace `let n: Number = ...` verifica sin coerciones. Si en cambio se escribiera `new Box[Number]("x")`, la sustitución haría que `"x": String` no conforme al `Number` esperado por el constructor, produciendo un `Mismatch` con el *span* del argumento. La invarianza impide, además, pasar un `Box[Dog]` donde se espera un `Box[Animal]`, aun cuando `Dog ≤ Animal`.

Comparativa. Java y Kotlin también usan *erasure*, con notación `<T>`; Scala y Nim emplean corchetes `[T]` como nuestra extensión. Rust, en cambio, *monomorfiza*: genera código especializado por instancia, más eficiente pero con mayor volumen de código [17,6]. La elección del *erasure* fue deliberada: encaja con una VM dinámicamente tipada sin requerir monomorfización, manteniendo el *backend* intacto.

4.2. Interfaces: contratos con conformidad nominal y estructural

Definición y sintaxis. Una interfaz declara un conjunto de firmas de métodos; puede heredar de otras (`extends`) y un tipo puede declarar que la implementa (`implements`), combinable con `inherits`:

(IFACE-NOM)	$\frac{T \text{ (o un ancestro) declara } \texttt{implements } J \quad (J = I \vee J \texttt{ extends}^* I)}{T \leq I}$
(IFACE-STR)	$\frac{\forall (m : (\overline{P}) \rightarrow R) \in \text{req}(I) \quad \exists (m : (\overline{P}') \rightarrow R') \in \text{decl}^*(T) : \overline{P}' = \overline{P} \wedge R' = R}{T \leq I}$

Figura 4. Conformidad a interfaz: nominal (`implements` transitivo) o estructural. $\text{decl}^*(T)$ son los métodos que T declara o hereda; `extends*` es la clausura transitiva de `extends`. La igualdad $\overline{P}' = \overline{P}$, $R' = R$ es **exacta** (invariante), tras sustituir los genéricos de I .

Listing 1.6. Interfaces, herencia múltiple entre interfaces y despacho dinámico.

```
interface Ord { compare(): Number; }
interface Eq { equals(): Boolean; }
interface Comparable extends Ord, Eq { min(): Number; }

type Person(name: String) implements Greeter {
  name: String = name;
  greet(): String => "hi, " @ self.name;
}
let g: Greeter = new Person("kevin") in print(g.greet()); // hi, kevin
```

Reglas semánticas. Las interfaces incorporan una pasada `check_interfaces` (entre `sign` y `check_overrides`) y un mapa `InterfaceInfo` en el contexto de tipos. Las reglas clave son:

- **Implementación obligatoria:** `T implements I` exige que T (o un ancestro) provea todos los métodos de I —y los heredados vía `extends`— con la firma exacta.
- **No instanciable:** `new Greeter()` es error; solo los tipos concretos se instancian.
- **implements solo sobre interfaces:** `type X implements Y` con Y concreto produce `NotAnInterface`.
- **Herencia múltiple** entre interfaces y **implementación múltiple** (`implements A, B`); las clases heredan de una sola.
- **Borrado en runtime:** el IR/VM no ven interfaces; el despacho dinámico existente resuelve los métodos por el `type_name` del objeto concreto, sin `vtable` separada.

Conformidad dual La función `conforms` acepta dos vías cuando el tipo destino es una interfaz. Sea I una interfaz y $\text{req}(I)$ el conjunto de firmas que exige, incluidas las heredadas por `extends`, con los parámetros genéricos de I sustituidos por los argumentos del sitio de uso. La Figura 4 da las dos reglas.

El emparejamiento **no es por nombre**: exige que cada método requerido exista con parámetros y retorno *exactamente* iguales (`own.params == req.params`

\&\& own.returns == req_ret en el código). Por tanto, un método con el mismo nombre pero distinto tipo de retorno *no* satisface la interfaz, y la conformidad falla; no hay, pues, riesgo de violar el principio de sustitución de Liskov [19], porque todo valor visto a través de *I* posee métodos con las firmas que *I* promete. El precio de esta exactitud es que la regla es **invariante**: rechaza retornos covariantes o parámetros contravariantes que serían seguros. Se eligió la igualdad exacta por simplicidad y por ser trivialmente sólida [23,1]; relajarla a subtipado es trabajo futuro. Esta dualidad sitúa a HULK entre el modelo nominal de Java/Kotlin/Swift y el estructural de Go/TypeScript.

Listing 1.7. Conformidad hacia una interfaz: primero la vía nominal, luego la estructural.

```
// dentro de conforms(a, b): cuando 'b' nombra una interfaz
if self.is_interface(&target_base) {
    if self.implements_interface(&cur_base, &target_base) {
        return true; // nominal: 'implements' (transitivo)
    }
    return self.structurally_implements(&cur_base, b); // estructural (A.10.2)
}
```

Implementación. El lexer aporta `interface/implements/extends`; el AST añade `InterfaceDecl` e `InterfaceMethodSig`, un campo `interfaces` en `Program` e `implements: Vec<TypeRef>` en `TypeDecl`; el parser, las reglas correspondientes; y la semántica, `lookup_interface_method`, `implements_interface`, `structurally_implements` e `interface_extends`. El IR y la VM permanecen intactos: las interfaces son borradas.

4.3. Recolector de basura *mark-and-sweep*

Motivación. La gestión previa, basada en `Rc<RefCell<Object>`, no podía liberar ciclos de objetos (el conteo de referencias nunca llega a cero en un ciclo). Para reclamarlos correctamente —y para ejercitar de primera mano un algoritmo clásico de gestión de memoria— se reemplazó por un recolector **mark-and-sweep** con un *heap* explícito.

Modelo de memoria. Los objetos viven en un `Heap` (un `Vec<Slot>`, con `Slot = Live(Object) | Free`) y se referencian por *handles* `ObjectId(u32)` que indexan el vector. Un `free_list` recicla los huecos liberados antes de crecer. El umbral de colección (`gc_threshold`, por defecto 1024) es configurable vía `HULK_GC_THRESHOLD`.

Listing 1.8. El *heap* gestionado por el GC.

```
pub struct Heap {
    slots: Vec<Slot>,           // Slot = Live(Object) | Free
    free_list: Vec<u32>,       // huecos reutilizables (ids
                               // reciclados)
    allocations_since_gc: usize,
    pub gc_threshold: usize,    // default 1024; HULK_GC_THRESHOLD
}
// Value::Object(ObjectId) reemplaza al antiguo Rc<RefCell<Object>>
```

Algoritmo.

1. **Mark:** recorrido en profundidad desde las raíces (la pila de operandos y los *scopes* de la VM). Todo `Value::Object(id)` se encola, se marca como vivo y se siguen recursivamente sus campos.
2. **Sweep:** los *slots* Live no marcados pasan a Free y su *id* se devuelve al `free_list` para reuso.
3. **Disparo:** tras cada asignación, si las allocaciones acumuladas alcanzan el umbral, se invoca `collect` con el conjunto de raíces.

Propiedades. El colector **reclama ciclos** (al partir de raíces vivas, un ciclo inalcanzable nunca se marca); **recicla slots** mediante el `free_list`; no compacta (el `Vec` crece monótonamente); es **backwards-compatible** (la salida observable es idéntica); y es **configurable** (`HULK_GC_THRESHOLD=1` fuerza GC por cada allocación, útil en pruebas). Las cadenas se gestionan por el *ownership* normal de Rust y no participan en este *heap*.

Traza de ejecución. El caso `tests/extension/gc.hulk` crea en cada vuelta de un bucle de cien iteraciones una caja efímera `Box(i)` que descarta de inmediato. Con `HULK_GC_THRESHOLD=8`, al alcanzar el umbral solo la caja de la iteración en curso es alcanzable desde las raíces; el *mark* no visita las anteriores y el *sweep* las recicla al `free_list`, manteniendo el *heap* acotado pese a las cien asignaciones. El conteo de referencias daría el mismo resultado en este caso acíclico; la diferencia se aprecia en `a.next := b; b.next := a`, donde solo el *tracing* reclama el ciclo.

Comparativa. CPython usa principalmente conteo de referencias con un detector de ciclos adicional al estilo Bacon–Rajan [4]; nuestra extensión se simplifica usando solo *tracing*. Lua emplea un mark-and-sweep incremental [13]; nosotros usamos la versión *stop-the-world* por claridad pedagógica. Java y Go usan colectores generacionales y concurrentes [15], fuera del alcance de un compilador educativo. Rust prescinde de GC mediante *ownership* estático [17]; HULK lo necesita porque permite mutación compartida a través de objetos. El algoritmo elegido es, en esencia, el que McCarthy introdujo para Lisp en 1960 [21]. La Tabla 5 contrasta las tres familias según los ejes que motivaron nuestra elección.

Cuadro 5. Comparación **cualitativa** (no empírica: no medimos *throughput* ni pausas) de familias de GC según los ejes que motivaron la elección. Las propiedades resumen lo establecido por Jones et al. [14,15].

Eje	Conteo de ref.	Mark-and-sweep	Generacional
Reclama ciclos	no (sin extra)	sí	sí
Pausas	incrementales	<i>stop-the-world</i>	cortas (menores)
Costo amortizado	proporcional a refs.	proporcional a lo vivo	favorece objetos jóvenes
Complejidad de impl.	baja	baja-media	alta
Ejemplo	CPython [4]	Lisp [21], Lua [13]	JVM, Go

El *mark-and-sweep* ofrece el mejor compromiso para nuestros fines: reclama ciclos (lo que el conteo de referencias no logra sin un detector adicional) con una complejidad de implementación abordable (a diferencia de los colectores generacionales o concurrentes), a cambio de pausas *stop-the-world* asumibles en un intérprete didáctico.

5. Evaluación, pruebas y calidad

El proyecto mantiene una batería de pruebas a varios niveles: *unit tests* dentro de cada módulo para la lógica privada; *integration tests* por *crate* para la API pública; y pruebas *end-to-end* que ejecutan programas HULK completos (la suite de la cátedra en `tests/hulk_std/` y la de la extensión en `tests/extension/`, con casos específicos `generics.hulk`, `interfaces.hulk` y `gc.hulk`). Cada variante de `SemError` cuenta con al menos una prueba negativa que la dispare, siguiendo el patrón `assert!(matches!(err, SemError::Variant ...))`. La integración continua (GitHub Actions) ejecuta *build*, pruebas, *clippy* y *rustfmt* en cada *push*, con `RUSTFLAGS="-D warnings"`, de modo que todo aviso del compilador se trata como error y solo se integran cambios con CI en verde. Esta disciplina hizo aflorar, por ejemplo, errores sutiles de ordenamiento en el despacho de métodos y en las raíces del GC, corregidos antes de la integración.

6. Trabajo relacionado

La arquitectura por fases sigue el modelo de Aho et al. [2] y Appel [3], y el *backend* de máquina de pila con *bytecode* sigue a *Crafting Interpreters* [22]. Las extensiones se apoyan en fuentes canónicas, ya citadas en sus secciones: la inferencia local en Pierce y Turner [24]; el subtipado nominal y estructural en Cardelli–Wegner [7], Abadi–Cardelli [1] y el principio de sustitución de Liskov–Wing [19]; el *erasure* de genéricos en Bracha et al. [6]; y la recolección *mark-and-sweep* en McCarthy [21] y Jones et al. [14]. Lo propio aquí no es ninguna de esas piezas, sino combinarlas —en particular la conformidad nominal *y* estructural— de forma coherente sobre una base modular en Rust.

7. Conclusiones

The Hitchhiker's Compiler implementa el pipeline completo de un compilador para HULK, desde el texto fuente hasta la ejecución, con un diseño modular que respeta SOLID y DRY, un análisis semántico basado en acumulación de errores y el patrón *poison*, y una máquina virtual con recolección de basura. Sobre esa base, el equipo diseñó, implementó y validó —mediante pruebas— tres extensiones (genericidad, interfaces y un recolector *mark-and-sweep*), acompañadas de reglas de tipado y de una comparación cualitativa con lenguajes conocidos.

Ventajas. La arquitectura de dependencias unidireccionales y el AST como contrato único hicieron las extensiones notablemente simples de implementar: genericidad e interfaces se añadieron *sin tocar* el IR ni la VM gracias al borrado de tipos, y el GC se aisló en un módulo `heap.rs`. El principio abierto/cerrado se cumplió en la práctica: casi todo el crecimiento fue mediante nuevas variantes de `enum` y nuevas pasadas, no modificando lógica existente. El manejo de errores por acumulación ofrece una experiencia de usuario superior a la de detenerse en el primer error.

Desventajas y críticas (declaradas abiertamente).

- **El *backend* es un intérprete, no un generador de código.** El ejecutable `./output` es un envoltorio que reejecuta el intérprete embebiendo la fuente validada; cumple el contrato de evaluación (que admite un *backend* de máquina virtual) pero no produce un binario nativo desde el IR. Lo decimos sin eufemismos.
- **Despacho sin optimizar.** Como se detalló, `CallMethod` resuelve por búsqueda en tabla a lo largo de la cadena de herencia, sin *vtables* ni cachés; los campos viven en *hash maps*. Es claro pero no rápido.
- **Inferencia local, no Hindley–Milner.** Sin unificación, la recursión mutua entre funciones sin anotar puede no resolverse al tipo ideal; el mínimo A.9 sí se cubre.
- **Conformidad estructural invariante.** Exige igualdad exacta de firmas; es sólida pero rechaza variancia segura.
- **GC *stop-the-world*** sin compactación (el *heap* crece de forma monótona); *erasure* sin *reified generics*; mensajes de error sintáctico menos específicos que los semánticos; sin comentarios de bloque.

7.1. Estudio comparativo con lenguajes de programación

Polimorfismo paramétrico (Tabla 6). La decisión central es *cómo* se realiza un parámetro de tipo. El **erasure** (HULK, Java [6,12], Scala/Kotlin sobre la JVM) mantiene una sola representación en ejecución: es barato y encaja con una VM dinámica, pero pierde la información de tipo en *runtime* (no hay *reified generics*, no se puede preguntar `x is List[Number]`). La **monomorfización** (C++, Rust [17]) genera una copia por instanciación: más rápida y type-safe, a costa

de tamaño de código. La **reificación** en el CLR de .NET [16] conserva los tipos en ejecución sin duplicar código, pero exige soporte del *runtime*. En *varianza*, HULK es **invariante** —como los genéricos crudos de Java—, la opción más conservadora y trivialmente sólida [23]; Scala, Kotlin y C# permiten varianza en el sitio de declaración, y Java la difiere al sitio de uso (*wildcards*). HULK elige el borrado e invarianza por la misma razón: son lo más simple que es sano, coherente con su *backend*.

Cuadro 6. Realización del polimorfismo paramétrico. Fuentes: [6,12,16,17,23].

Lenguaje	Realización	Varianza	Tipo en <i>runtime</i>
HULK ext.	<i>erasure</i>	invariante	borrado
Java	<i>erasure</i>	uso (<i>wildcards</i>)	borrado
Scala / Kotlin	<i>erasure</i> (JVM)	declaración	borrado
C# (.NET)	reificación	declaración	presente
C++ / Rust	monomorfización	invariante	especializado

Subtipado e interfaces (Tabla 7). El contraste clásico es **nominal** frente a **estructural**. La disciplina nominal (Java, C#, Swift) hace que un tipo conforme a una interfaz solo si lo *declara*: evita coincidencias accidentales y produce diagnósticos claros, al precio de exigir que el autor del tipo conozca la interfaz de antemano. La estructural (Go, TypeScript [5]) deduce la conformidad de la forma, favoreciendo el desacoplamiento, pero admite coincidencias no intencionadas. HULK ofrece **ambas** (Figura 4): **implements** explícito o igualdad exacta de firmas. La igualdad exacta es la diferencia fina y deliberada frente al subtipado de métodos de Abadi–Cardelli [1]: renunciamos a la variancia de métodos (retorno covariante, parámetro contravariante) a cambio de una regla que es sólida por construcción respecto del principio de sustitución [19].

Gestión de memoria (Tabla 8). El espacio va del control manual (C/C++) a la ausencia de GC por *ownership* estático en Rust [17], pasando por las dos familias automáticas: conteo de referencias y rastreo [14]. CPython usa conteo con un detector de ciclos añadido [4]; HULK usa solo rastreo *mark-and-sweep* [21], que reclama ciclos sin estructura extra a cambio de pausas *stop-the-world*. Los

Cuadro 7. Disciplina de conformidad a interfaces/protocolos. Fuentes: [7,1,10,5,19].

Lenguaje	Disciplina	Regla de conformidad
HULK ext.	nominal y estructural	implements , o firmas exactas
Java / C# / Swift	nominal	declaración explícita
Go	estructural	posee los métodos requeridos
TypeScript	estructural	forma compatible (con variancia)

runtimes de producción (JVM, Go, .NET) añaden recolección **generacional** — basada en la hipótesis de que la mayoría de los objetos mueren jóvenes [27]— y concurrencia para acortar las pausas. HULK se queda en el escalón más simple que es correcto: prioriza entender el algoritmo sobre minimizar la latencia, una elección adecuada a un intérprete docente pero explícitamente inferior en rendimiento.

Cuadro 8. Estrategias de gestión de memoria. Fuentes: [14,21,4,27,17,13].

Lenguaje	Estrategia	Reclama ciclos	Pausas
HULK ext.	<i>mark-and-sweep</i> (STW)	sí	sí (STW)
Lisp / Lua	rastreo (m-s / incremental)	sí	sí / incrementales
CPython	conteo + detector de ciclos	sí (con detector)	incrementales
JVM / Go / .NET	generacional / concurrente	sí	cortas
Rust	<i>ownership</i> estático	n/a (sin GC)	ninguna
C / C++	manual	n/a	ninguna

7.2. Trabajo futuro

Las líneas de mejora incluyen: (i) un *backend* de generación de código a C o LLVM, sustituyendo el intérprete; (ii) varianza acotada (**covariant/contravariant**) y cotas en los parámetros de tipo, superando la invarianza actual; (iii) un GC generacional o incremental para reducir las pausas; (iv) diagnósticos sintácticos con recuperación de errores y sugerencias; y (v) *reified generics* opcionales para habilitar reflexión de tipos en *runtime*. En conjunto, el trabajo cumple su objetivo docente: un compilador correcto y comprobable para HULK y tres extensiones con reglas de tipado precisas, cuya incorporación incremental ilustra el valor práctico de un diseño modular.

Disponibilidad. El código fuente completo, las pruebas y la documentación están disponibles en el repositorio del proyecto [26].

Referencias

1. Martín Abadi and Luca Cardelli. *A Theory of Objects*. Monographs in Computer Science. Springer, New York, 1996.
2. Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.
3. Andrew W. Appel. *Modern Compiler Implementation in ML*. Cambridge University Press, Cambridge, 1998.
4. David F. Bacon and V. T. Rajan. Concurrent cycle collection in reference counted systems. *Lecture Notes in Computer Science*, 2072:207–235, 2001.

5. Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding TypeScript. In *ECOOP 2014 – Object-Oriented Programming*, volume 8586 of *Lecture Notes in Computer Science*, pages 257–281. Springer, 2014.
6. Gilad Bracha, Martin Odersky, David Stoutamire, and Philip Wadler. Making the future safe for the past: Adding genericity to the Java programming language. In *Proceedings of OOPSLA '98*, pages 183–200. ACM, 1998.
7. Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.
8. George E. Collins. A method for overlapping and erasure of lists. *Communications of the ACM*, 3(12):655–657, 1960.
9. Frank DeRemer and Thomas Pennello. Efficient computation of LALR(1) look-ahead sets. *ACM Transactions on Programming Languages and Systems*, 4:615–649, 1982.
10. Alan A. A. Donovan and Brian W. Kernighan. The Go programming language, 2015.
11. Universidad de La Habana Facultad de Matemática y Computación. HULK — the book: Havana university language for kompilers, 2024. Material docente del curso de Compilación.
12. James Gosling, Bill Joy, Guy Steele, Gilad Bracha, and Alex Buckley. *The Java Language Specification, Java SE 8 Edition*. Addison-Wesley, 2014.
13. Roberto Ierusalimsky. *Programming in Lua*. Lua.org, 4th edition, 2016.
14. Richard Jones, Antony Hosking, and Eliot Moss. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, Boca Raton, FL, 2011.
15. Richard Jones and Rafael Lins. *Garbage Collection: Algorithms for Automatic Dynamic Memory Management*. John Wiley and Sons, Chichester, 1996.
16. Andrew Kennedy and Don Syme. Design and implementation of generics for the .NET common language runtime. In *Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation (PLDI '01)*, pages 1–12. ACM, 2001.
17. Steve Klabnik and Carol Nichols. *The Rust Programming Language*. No Starch Press, San Francisco, CA, 2nd edition, 2023.
18. Donald E. Knuth. On the translation of languages from left to right. *Information and Control*, 8(6):607–639, 1965.
19. Barbara H. Liskov and Jeannette M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16:1811–1841, 1994.
20. Niko Matsakis and Contributors. LALRPOP: LR(1) parser generator for Rust, 2024. <https://github.com/lalrpop/lalrpop>.
21. John McCarthy. Recursive functions of symbolic expressions and their computation by machine, part I. *Communications of the ACM*, 3(4):184–195, 1960.
22. Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021.
23. Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, Cambridge, MA, 2002.
24. Benjamin C. Pierce and David N. Turner. Local type inference. *ACM Transactions on Programming Languages and Systems*, 22(1):1–44, 2000.
25. Maciej Płesak and Contributors. Logos: Create ridiculously fast lexers, 2024. <https://github.com/maciejhirsz/logos>.
26. Kevin Alejandro Torres Perera, Lianny de la Caridad Revé Valdivieso, and Jocdan Lismar López Mantecón. The hitchhiker's compiler, 2026. <https://github.com/MatCom-Developing-Team-10/The-Hitchhikers-Compiler>.

27. David Ungar. Generation scavenging: A non-disruptive high performance storage reclamation algorithm. In *Proceedings of the ACM SIGSOFT/SIGPLAN Software Engineering Symposium on Practical Software Development Environments*, pages 157–167. ACM, 1984.